

EE 7260: Applied Big Data Engineering

Mini Project Report

Smart City Traffic & Congestion System

GitHub Link: <https://github.com/pabasara-samarakoon-4176/smartcity-kafka>

YouTube Demo: https://youtu.be/_2iLcSAhDys

Group Members

Rupasinghe A.A.H.S EG/2020/4168

Samarakoon P.G.C EG/2020/4176

Smart City Traffic & Congestion System

1. Overview

This project is a real time Smart City Traffic and Congestion Monitoring System for the city of Colombo in Sri Lanka. The Python producer generates data, mimicking the 16 Colombo road junctions, including Pettah, Fort, Maradana, Rajagiriya, Peliyagoda, and outputs vehicle count and average speed data every second. It is consumed by the pipeline through Apache Kafka, processed in real-time using Apache Spark Structured Streaming to calculate a Congestion Index and a critical speed alert, stored as partitioned Parquet files, and executes a batch job every night using Apache Airflow to generate a report of the Peak Traffic and Intervention in CSV format.

The solution is based on a Lambda Architecture with two layers: a Speed Layer, which is responsible for providing alerts in sub-second time, and a Batch Layer, which runs daily for analytical reporting. All of them are containerized using Docker Compose and comprise eight services: Zookeeper, Kafka, Kafdrop, Spark Master, Spark Worker, Airflow Scheduler, Airflow Webserver, and a PostgreSQL metadata database.

Architecture Diagram

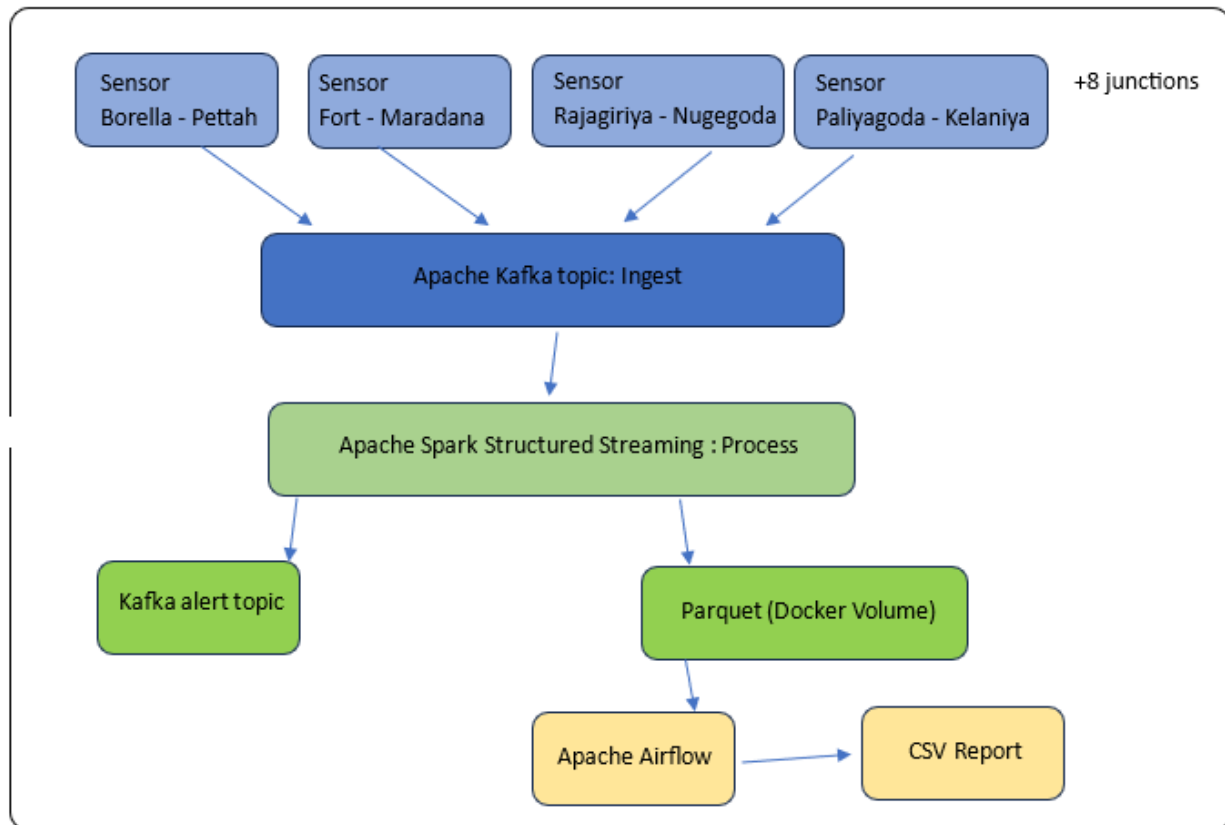


Figure 1: The Architecture Flow

2. Justification of Tools Selected

Table 1: Technology stack summary

Tool	Role
Apache Kafka	Event ingestion and alert transport
Apache Spark	Stream processing and batch reporting
Apache Airflow	Nightly batch orchestration
Parquet + Docker Volume	Analytics storage sink
PostgreSQL 14	Airflow metadata database

2.1 Apache Kafka

Kafka was chosen as the ingestion layer because it can handle sixteen sensor streams at once and needs to be durable, ordered, and fault-tolerant. Using `sensor_id` as the message key makes sure that all readings for a given junction go to the same partition and are read in the right order. This is important for accurate windowed aggregations. There are two listeners set up: one for Spark containers that is internal and one for the Python producer that is external. A dedicated critical-traffic topic serves as the alert sink, allowing downstream consumers to subscribe without having to change the Spark job. RabbitMQ was turned down because it didn't have a persistent, replayable log, which meant that a consumer who restarted would miss events that happened while it was down.

2.2 Apache Spark Structured Streaming

There were three reasons why Spark was chosen over Storm. First, the same cluster works for both the streaming job and the nightly batch report because Spark treats both as the same DataFrame API. Storm does not have batch infrastructure. The Congestion Index (total vehicles divided by window average speed, with a small epsilon to avoid division by zero) is a single DataFrame expression. Storm would need a custom bolt. Third, the Parquet writer that comes with Spark's native partition-by feature makes storage that the batch layer can query right away without any extra ETL.

2.3 Apache Airflow

We chose Airflow because the 04:00 daily report is a scheduled workflow that knows about dependencies and is good for DAG orchestration. The DAG uses `SparkSubmitOperator` to send the batch job to the Spark master. The `catchup` option is set to false so that backfilling doesn't happen on the first deployment. A PostgreSQL-backed metadata store keeps task history and logs even when containers are restarted. A simple cron script was turned down because it didn't have retry logic, operational logging, or the visual DAG inspector, all of which are necessary for a production traffic system.

2.4 Parquet on Shared Docker Volume

We chose Parquet over a relational database because the batch report only reads a few columns from potentially millions of rows. Parquet's columnar layout only reads the columns that are needed, unlike row-based formats. Files are split up by sensor ID and traffic date, so the nightly job only reads today's directory

through partition pruning. Airflow only uses PostgreSQL as its metadata store. People thought about using Cassandra to store alerts, but they decided against it because the critical-traffic Kafka topic already has a durable, subscribable alert stream that doesn't need a NoSQL cluster.

3. Event Time vs. Processing Time

In each sensor event, two time stamps coexist, and it is essential to recognize them correctly to ensure the accuracy of the congestion index. Event time is the time stamp that is present in the sensor JSON payload, which is the actual time that the number of vehicles and the speed measurement were physically recorded at the junction. This is created by the producer as a 'simulated timestamp' that will map the current real minute of the day to a 24-hour simulated traffic cycle, meaning the entire diurnal pattern will be applied across a 60-minute test run: morning rush, midday, evening rush, and night.

Processing time is the wall-clock time when Kafka receives the message or when Spark reads the message from the topic. This is different from event time when containers are restarted, when there is network congestion, or when there is Spark micro batch backpressure.

All windowed aggregations are performed on event time in the stream processor. The Kafka broker receipt timestamp is stored as an independent audit column and is not used in the window calculation. All grouping and aggregation is done based on the parsed sensor event time stamp. This is important because in the event of congestion, a reading taken during a congestion episode could arrive at the Spark consumer several seconds after the reading was taken. Processing time would be used to end up in the wrong one-minute window, which would cause a silent distortion of the Congestion Index for that junction and could delay or suppress a critical alert.

In order to manage late events, a 2-minute watermark is introduced before the period of windowing. This makes Spark wait up to two minutes after the end of a window before completing and presenting its result. Events that are more than two minutes late are ignored. Otherwise, Spark would keep all windows open forever, allowing for an arbitrary amount of delay for events, and would eventually use an infinite amount of memory. The tolerance of 2 minutes is suitable for the simulated environment; a realistic IoT installation on a 4G or LoRaWAN network would require a larger tolerance of 5 or 10 minutes because of the variability of the network.

The five-minute window proposed in the assignment brief was not used for the one-minute window of tumbling. The producer maps each real-time minute to one simulated hour, meaning that a one-minute window in the real-time stream will be matched to a one-hour window in the simulated traffic stream, thereby eliminating the need for an extra transformation step to match the hourly aggregations that the producer calculates in the nightly batch report.

4. Ethics and Data Governance

4.1 Privacy Implications

However, this implementation does not track any personally identifiable information or average speeds, but would still have privacy and civil liberties implications in any real-life deployment of the Smart City sensor scenario.

There are often traffic sensors at actual junctions where the traffic cameras are also deployed in the hope of capturing the registration plates of vehicles, as well as, in some configurations, driver biometrics. Even if it were based solely on flow data and no other sources, inference attacks are still possible: junction-level readings, when combined with external data like vehicle registration or mobile network cell-tower readings, can allow inference of a single user's movement pattern over time. Nothing in the EU GDPR or Sri Lanka's Personal Data Protection Act (2022) indicates that the inferred patterns would not constitute indirect personal data, since they would relate to the behavior of a natural person and their location, which would activate the full data protection obligations even if there was no name or plate number on the dataset.

3.2 Data Governance Recommendations

- **Data Minimization:** Only the number of vehicles and the average speed of the vehicles must be recorded by sensors. Any plate recognition, image capture, and biometric functionality should be physically disabled and contractually prohibited at procurement. This is by design in the current synthetic implementation; it needs to be enforced for real hardware deployment.
- **Purpose Limitation:** The Kafka topics and Parquet archive should be accessible only to traffic management personnel who have been authorized. Kafka ACLs and file-system permissions can't be opened to law enforcement or any other agency without a valid court order.
- **Retention Policy:** The following Kafka events should be held for 7 days: raw events. Parquet aggregates need to be kept for 30 days. Daily CSV reports can be stored for a year. An Automated Airflow DAG should ensure that expired partitions are deleted. Current implementation does not contain a purge mechanism and will need this addition prior to deployment to production.
- **Transparency:** Citizens should be made aware of the location of the sensors, the type of data collected, the length of time it is retained, and who controls the data, by notices at each sensor location and by a transparency portal that is accessible to the public.
- **No Re-identification:** All data provided to third parties, like urban planners or researchers, should be aggregated at least hourly and should not contain reading level timestamps, which could allow for reconstruction of the individual journeys.